



DEPARTAMENTO DE INGENIERÍA ELÉCTRICA
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE CONCEPCIÓN
CONCEPCIÓN, CHILE.



549305 - Taller de Aplicación TIC

Miniproyecto 2

Profesor: Vincenzo Caro Fuentes

Integrantes: Oscar I. Ortiz Molina

Joaquín J. Mardones Gallegos

31 de mayo de 2026

Índice

1. Introducción	2
2. Desarrollo del Sistema PokéDuel	3
2.1. Prueba de Concepto: Comunicación Ping-Pong Serial	3
2.2. Arquitectura y Hardware: Topología de Red I2C	3
2.3. Adquisición y Normalización de Datos (PokéAPI)	4
2.4. Motor de Combate e Inteligencia Artificial	5
2.5. Interfaz Gráfica y Protocolo de Sincronización	6
2.6. Modificaciones y Valor Agregado	8
3. Conclusión	10

1. Introducción

En este informe presentamos el desarrollo de "PokéDuel", un sistema distribuido interactivo diseñado para el Miniproyecto 2 de la asignatura. El objetivo fundamental de esta etapa fue lograr la convergencia entre el hardware y el software mediante el diseño de una arquitectura escalable, integrando microcontroladores, interfaces gráficas complejas y adquisición automatizada de datos remotos.

El proyecto se estructuró dividiendo las responsabilidades lógicas. Por un lado, se desarrolló un robusto simulador de combates en Python (usando PyQt6) que actúa como terminal de usuario y motor matemático. Por otro, se construyó un puente de comunicación de hardware enlazando placas Arduino mediante el protocolo I2C para gestionar el flujo de datos multijugador. Para dotar al ecosistema de información verídica, se implementó un algoritmo de *Web Scraping* concurrente que descarga y normaliza datos directamente desde una API REST oficial.

A lo largo de este documento, detallaremos la evolución ingenieril del proyecto: desde la prueba de concepto para enlazar los puertos seriales sin bloquear la interfaz gráfica, pasando por la validación de control de calidad (QA) del motor matemático, hasta llegar a la integración de elementos multimedia y protocolos de sincronización asíncrona que otorgan una experiencia fluida al usuario final.

2. Desarrollo del Sistema PokéDuel

La arquitectura del sistema fue diseñada bajo un enfoque de separación de responsabilidades, dividiendo el proyecto en tres capas fundamentales: el motor lógico (*Backend*), la interfaz de usuario (*Frontend*) y el puente de comunicación física (*Hardware*).

2.1. Prueba de Concepto: Comunicación Ping-Pong Serial

Antes de programar la lógica del juego, era indispensable garantizar la estabilidad de la transmisión de datos. Se desarrolló un script aislado (`ping_pong.py`) con una interfaz minimalista para validar el enlace bidireccional. Este módulo permite seleccionar el puerto COM y enviar una trama inicial "PING". Utilizando un hilo secundario (`QThread`), el sistema lee el puerto en tiempo real, detecta la trama entrante y responde automáticamente con "PONG". Esto demostró la viabilidad de la comunicación asíncrona, evitando el congelamiento (*freeze*) de la interfaz gráfica.

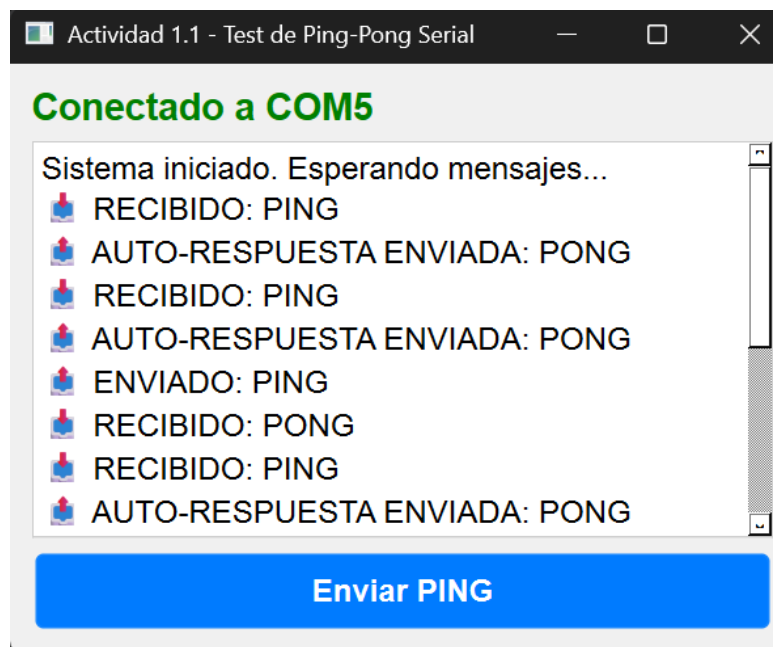


Fig. 1: Interfaz de validación del enlace Serial (Ping-Pong).

2.2. Arquitectura y Hardware: Topología de Red I2C

Para establecer la conexión entre ambos jugadores, las placas Arduino UNO fueron interconectadas utilizando el protocolo I2C mediante la librería `Wire.h`. El esquema físico utiliza una topología bidireccional enlazando los pines SDA (A4) y SCL (A5), compartiendo además la referencia de tierra (GND).

A nivel de firmware, el código en C++ implementa un sistema de *polling* que captura las cadenas de texto enviadas por Python a través del puerto Serial y las retransmite inmediatamente al otro dispositivo por el bus I2C (hacia la dirección configurada), cerrando el ciclo de comunicación de extremo a extremo.

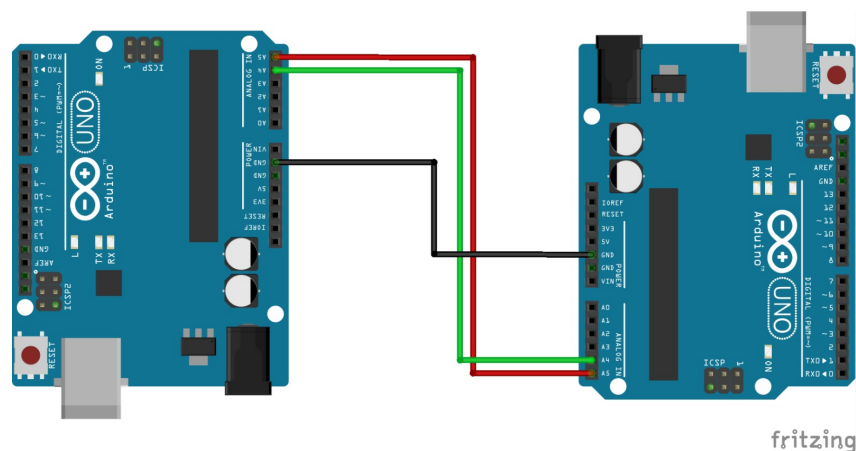


Fig. 2: Esquemático de la topología I2C entre ambos microcontroladores.

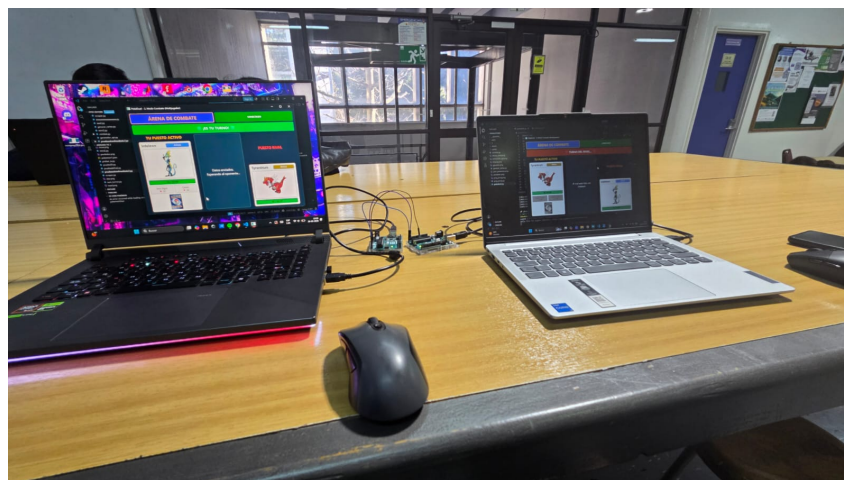


Fig. 3: Montaje físico del sistema operando en el entorno de pruebas.

2.3. Adquisición y Normalización de Datos (PokéAPI)

Para alimentar el juego con cartas reales, se programó el script `generar_cartas.py`. Este módulo se conecta a una API REST externa (PokéAPI) para descargar la base de datos completa de la franquicia. Para optimizar el tiempo de respuesta, se implementó procesamiento concurrente (*Multithreading*) utilizando `ThreadPoolExecutor`, logrando realizar múltiples peticiones simultáneas.

El algoritmo somete los datos crudos a un filtrado riguroso: ignora los ataques que no causan daño directo, normaliza los textos internos (removiendo guiones y aplicando mayúsculas) ante la ausencia de traducciones al español, y escala matemáticamente el daño de los ataques (multiplicándolos por un factor de 0.4) para restringirlos al rango de 15 a 45 estipulado en los requerimientos.

```

99 if __name__ == "__main__":
100     for carpeta in ["cartas", "assets"]:
101         if not os.path.exists(carpeta):
102             os.makedirs(carpeta)
103
104     lista_pokemon = obtener_lista_completa()
105     print(f"Se obtuvieron {len(lista_pokemon)} Pokémon. Iniciando limpieza de te:
106
107     completados = 0
108     with ThreadPoolExecutor(max_workers=15) as executor:
109         futuros = {executor.submit(extraer_datos_pokeapi, poke): poke for poke in
110         for futuro in as_completed(futuros):
111             completados += 1
112             resultado = futuro.result()
113             print(f"[{completados}/{len(lista_pokemon)}] {resultado}")
114
115     print("\n🚀 |Extracción y formateo completado!")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

[375/1025] ✅ Groudon procesado.
[376/1025] ✅ Regirock procesado.
[377/1025] ✅ Metang procesado.
[378/1025] ✅ Huntail procesado.
[379/1025] ✅ Regice procesado.
[380/1025] ✅ Rayquaza procesado.
[381/1025] ✅ Latios procesado.
[382/1025] ✅ Deoxys-normal procesado.
[383/1025] ✅ Jirachi procesado.
[384/1025] ✅ Kyogre procesado.
[385/1025] ✅ Turtwig procesado.

```

Fig. 4: Consola ejecutando el *scraping* masivo concurrente.

```

cartas > {} wobbuffet.json > ...
1 {
2     "name": "Wobbuffet",
3     "type": "psychic",
4     "hp": 190,
5     "attacks": [
6         {
7             "name": "Placaje",
8             "damage": 20
9         },
10        {
11            "name": "Golpe Fuerte",
12            "damage": 35
13        }
14    ],
15    "image_path": "assets/wobbuffet.png"
16 }

```

Fig. 5: Estructura JSON filtrada y normalizada para el motor.

2.4. Motor de Combate e Inteligencia Artificial

El *backend* lógico (*combat.py*) fue programado para calcular dinámicamente las debilidades y resistencias. Alejándose del esquema básico de tres elementos, se implementó la matriz oficial completa de los 18 tipos de Pokémon. En el Modo Entrenamiento, la inteligencia artificial genera un oponente ("Dummy") que asume la identidad y tipos de un Pokémon real aleatorio, pero restringe estrictamente su HP (80-150) y daño para adherirse a la pauta.

Fase de QA (Control de Calidad): Antes de escalar el sistema a las 1025 criaturas, se ejecutó una rigurosa prueba de estrés manual. Se seleccionó un grupo táctico de 20 Pokémon específicos con características complejas (como Rayquaza, Magnezone, Mimikyu, Gengar, entre otros) para validar que los multiplicadores de daño por tipos duales, debilidades, e inmunidades operaran a la perfección. Una vez validada esta fase sin errores lógicos, se procedió a la descarga masiva.



Fig. 6: Ejemplo visual del multiplicador de daño en tiempo real, generado por el motor lógico de combate.

2.5. Interfaz Gráfica y Protocolo de Sincronización

El *frontend* (`pokeduel.py`) presenta un "Estadio Pokémon" diseñado con hojas de estilo (CSS) en PyQt6. Destacan detalles inmersivos como el botón de relevo, rediseñado dinámicamente utilizando el dorso clásico de una carta. Al accionar este relevo, se despliega un menú interactivo que asiste al jugador informando el estado de salud restante del equipo para la toma de decisiones tácticas.

El flujo multijugador se gestiona mediante un protocolo propio de sincronización (*handshake*). Al establecerse la conexión, se intercambian tramas de inicialización (ej. `READY|Nombre`), y durante la partida se envían comandos estructurados (ej. `ATK|Ataque|Daño` o `SWAP_FORZADO`).

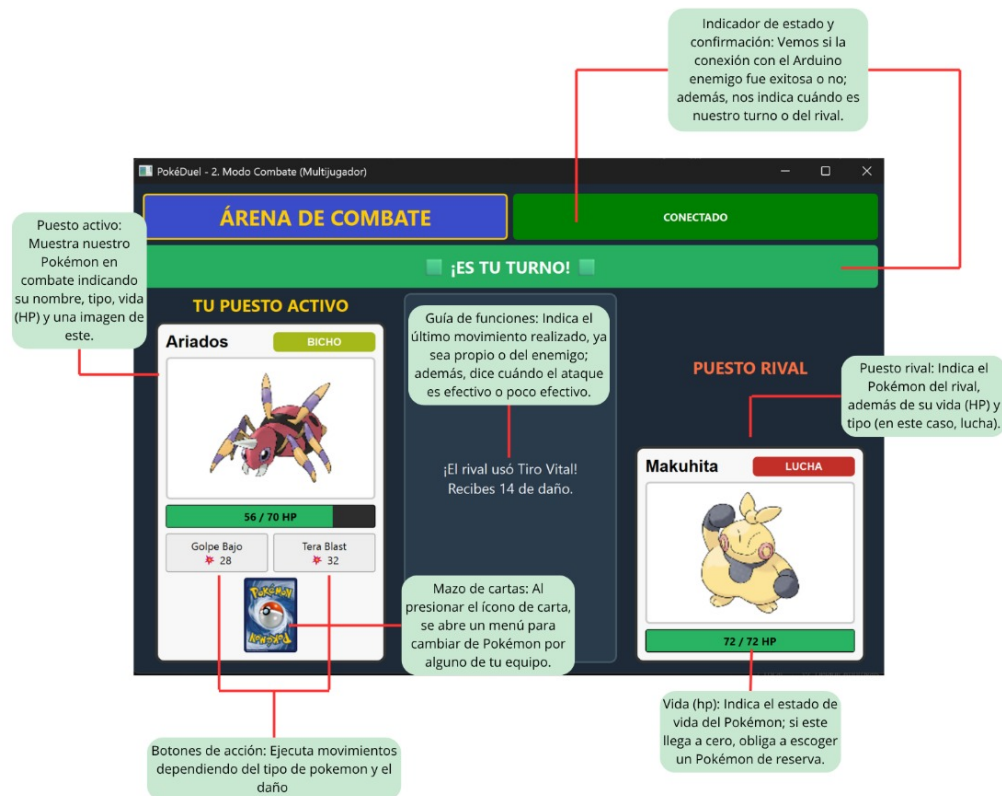


Fig. 7: Mockup de la arena de combate en estado inicial.

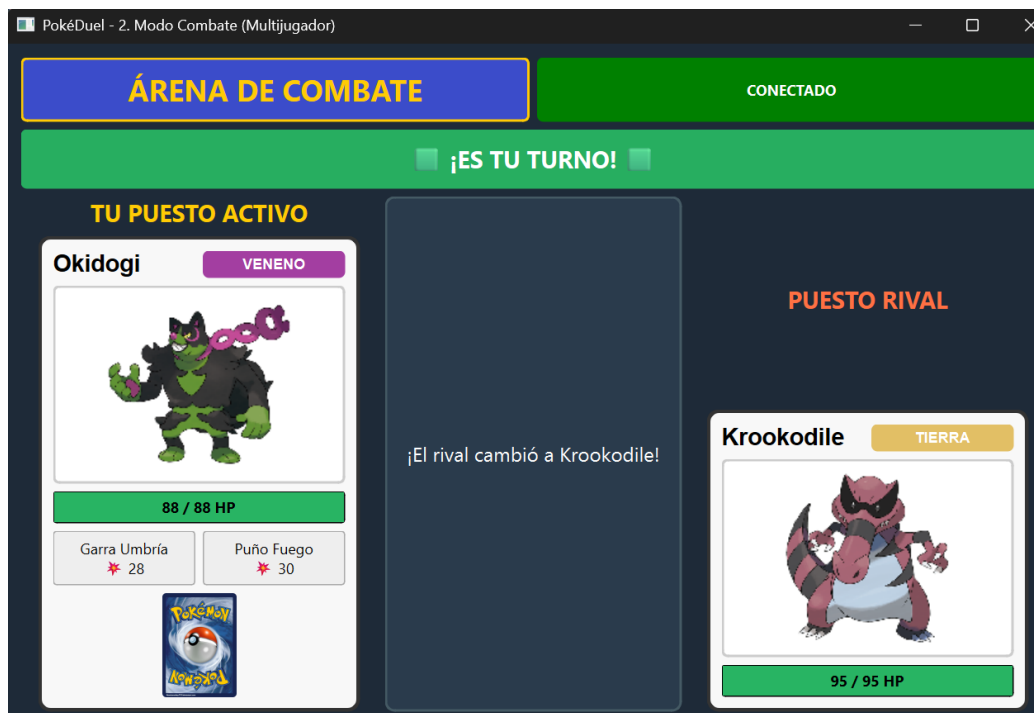


Fig. 8: Interfaz gráfica en plena ejecución de combate, registrando ataques y daños en el panel central.

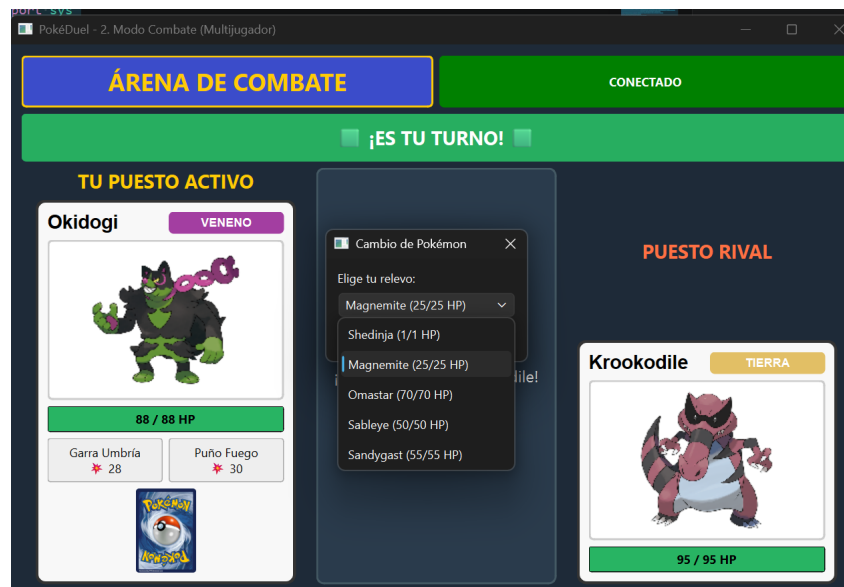


Fig. 9: Menú modular de relevo táctico mostrando el equipo disponible y sus puntos de vida actuales.

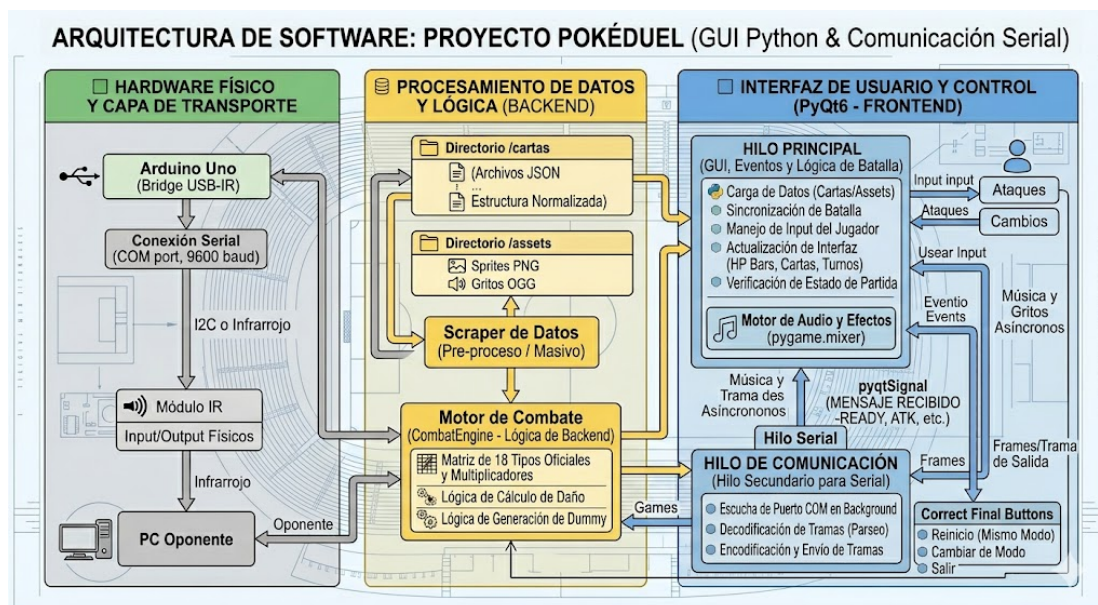


Fig. 10: Diagrama de bloques de la arquitectura de red y el flujo de la información asíncrona.

2.6. Modificaciones y Valor Agregado

Para dotar al sistema de un estándar profesional, se incorporaron elementos adicionales:

- **Motor Multimedia:** Mediante la librería `pygame.mixer`, se integró una banda sonora competitiva en bucle y se programó la descarga automática de los gritos de batalla nativos (.ogg) de cada Pokémon. Estos efectos se sincronizan asíncronamente con los eventos de la interfaz (aparición y derrota).
- **Traducción Dinámica:** El sistema traduce en tiempo real los metadatos en inglés provenientes de la API, renderizando etiquetas (*badges*) de colores en español (ej. VENENO, ELÉCTRICO).

para cada tipo.

- **Gestión de Fin de Partida:** Al concluir un encuentro, un sistema modular (QDialog) despliega el resultado de victoria o derrota. Destaca el botón "Jugar Otra Vez (Mismo Modo)", que limpia la mesa, regenera equipos aleatorios y reinicia el combate inmediatamente sin cerrar la conexión I2C.



Fig. 11: Menú modular tras obtener la victoria.

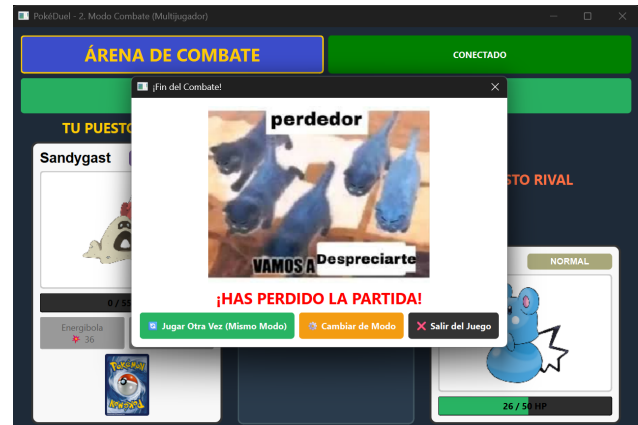


Fig. 12: Menú modular tras perder la partida.

3. Conclusión

El desarrollo del sistema PokéDuel representó un enorme desafío de ingeniería. A diferencia de un software con un entorno de ejecución local estático, coordinar de manera fluida una interfaz gráfica de alta resolución, la manipulación de datos remotos masivos y el envío de tramas a través de hardware físico implicó resolver múltiples y complejos problemas de concurrencia y sincronización.

El mayor obstáculo que enfrentamos durante estas semanas fue establecer la conexión estable en la etapa inicial de prueba (Ping-Pong). Sincronizar los puertos seriales para que Python pudiera escuchar los datos entrantes continuamente sin congelar la ventana gráfica, sumado a la correcta gestión de los tiempos de lectura y escritura en el bus I2C entre ambos Arduinos, nos tomó numerosos intentos, pruebas y *debugging*. Superar esta etapa nos enseñó la necesidad ineludible de utilizar hilos de ejecución independientes (**QThread**) para separar estrictamente los procesos de red del renderizado visual.

Por otro lado, la modularidad del código fue clave para el éxito del proyecto. Aislar el motor matemático en un archivo independiente no solo nos permitió expandir la matriz de tipos a los 18 elementos oficiales, sino que facilitó enormemente realizar un estricto control de calidad (QA) manual con 20 Pokémon tácticos, antes de escalar el sistema y confiar en la automatización hacia la base de datos completa de 1025 criaturas de la PokéAPI.

En definitiva, estamos enormemente satisfechos con los resultados obtenidos. Logramos consolidar y aplicar conocimientos de telecomunicaciones, algoritmos de *scraping*, diseño orientado a objetos y protocolos I2C, para construir una plataforma robusta, inmersiva y altamente funcional.